AskTheDatabase: Overcoming Exploration Collapse in Multi-Turn Relational RLVR via Procedural Distillation and Compound Verification

Christian Contreras Campana [1]

Abstract

Deploying Large Language Models (LLMs) as autonomous relational database interfaces requires agents that can iteratively explore database catalogs, execute trial queries, and recover from runtime operational exceptions. In zero-knowledge environments, applying multi-turn Reinforcement Learning with Verifiable Rewards (RLVR) directly to conventional single-turn supervised fine-tuned (SFT) models causes catastrophic **exploration collapse**: parallel rollouts fail simultaneously on unfamiliar schemas, driving intra-group advantage variance to zero ($\mathrm{std}(R) \to 0 \implies A_i \to 0$) and causing the policy gradient in Group Relative Policy Optimization (GRPO) to vanish (stalling for 27 steps and collapsing to 1.26% Test-Suite accuracy on Spider Dev). In this work, we present **AskTheDatabase**, an integrated framework structured across three core technical stages: (1) **Harness-Zero Procedural Distillation**, where an external procedural reviewer ($\mathcal{K}_{\text{SQL}}$) monitors and repairs intermediate exploratory actions mid-flight with *zero ground-truth SQL leakage*, internalizing catalog exploration habits directly into model parameters; (2) **Procedurally Grounded Multi-Turn RLVR**, which bootstraps non-zero reward variance by step 3 ($9.0\times$ faster), maintains high intra-group variance ($\mathrm{std}(R) = 0.443$), sustains a $+76.95$ cumulative AUC advantage, and yields a $10.7\times$ relative gain in held-out Spider Dev Test-Suite accuracy (13.54%) alongside a $15.2\times$ higher autonomous error recovery rate (3.04% vs. 0.20%); and (3) **Compound Verification**, proving that test-time verification operates orthogonally to procedural grounding—excelling at re-ranking static candidate queries when full schemas are pre-injected, but requiring an exploratory, grounded generator when operating autonomously. Finally, we establish a **Dual-Regime Evaluation Framework**: while procedurally grounded RLVR dominates in the primary zero-knowledge autonomous regime (13.54% vs. 1.26% TS, a $10.7\times$ advantage), evaluating under static single-turn full-schema harnesses reveals an informative specialization trade-off—conventional SFT achieves 48.84% TS with test-time verification, while the multi-turn agent pays a prompt-format tax (37.72% TS) due to its ingrained refusal to guess without schema inspection. The framework is validated via an in-memory **Zero-Budget Evaluation Micro-Harness** executing in 126.8 ms on CPU with perfect monotonic rank correlation ($\rho = 1.000$) to Spider.

1. Introduction

Natural language-to-SQL (Text-to-SQL) semantic parsing serves as the foundational bridge between non-technical domain practitioners and relational database management systems (RDBMS) (Yu et al., 2018; Li et al., 2023). Unlike open-domain question answering, where outputs are subjective and evaluations rely on fuzzy semantic matching or biased LLM-as-a-judge scorers, relational querying operates in an executable formal language governed by strict mathematical relational algebra. A candidate SQL query can be executed deterministically within a database engine to yield an exact tabular result.

However, aligning neural language models for autonomous relational execution in realistic, zero-knowledge environments exposes three fundamental, interlocking research challenges:

1. **Exploration Collapse in Multi-Turn RLVR**: Single-turn models frequently hallucinate column names, infer invalid foreign keys, or generate unindexed Cartesian cross-joins. When training multi-turn agents from conventional SFT baselines using Group Relative Policy

[1]Independent Researcher, Hamburg, Germany. Correspondence to: Christian Contreras Campana <chrisjcc.physics@gmail.com>.

Preprint. October 31, 2026.

Optimization (GRPO) (Shao et al., 2024), ungrounded rollouts fail simultaneously on complex unseen schemas. This collapses intra-group reward variance ($\mathrm{std}(R) \to 0$), driving advantage estimates to zero ($A_i \to 0$) and causing the policy gradient to vanish, trapping the policy in an extended exploration plateau.

2. **The Candidate Pool Bottleneck in Test-Time Verification**: In production deployments, ground-truth SQL (gold_sql) is unavailable. While test-time tournament verification (e.g., Best-of-G selection via LLM-as-a-Verifier) can arbitrate between competing queries, a verifier acts purely as an evaluation filter. This motivates a central research question: *does test-time verification compensate for an ungrounded generator policy, or does verification compound value exclusively when paired with a procedurally grounded generator policy?* If an ungrounded generator policy samples candidates that all hallucinate non-existent columns, the verifier cannot synthesize correct queries from an empty candidate pool (Pass@$G \to 0$).
3. **The Evaluation Latency Bottleneck**: Full cross-domain benchmark evaluation (e.g., 1,034 queries on Spider Dev) requires 180–350 seconds of cluster GPU compute over disk-bound SQLite files, preventing instant developer feedback during interactive policy alignment and automated regression testing.

Conceptual Evolution: From Static Parsers to Autonomous Agents. To address these challenges, our investigation traced three conceptual milestones:

- **The Starting Point (Single-Turn Paradigm)**: Text-to-SQL benchmarks historically assumed the complete schema DDL fits inside a single prompt context, training models as static one-shot semantic parsers mapping $\langle \text{DDL} + \text{Question} \rangle \to \text{SQL}$ without database interaction.
- **The Reframing (Autonomous Multi-Turn Agent)**: In enterprise databases with hundreds of tables, complete schemas cannot fit into prompt contexts. We reframed the task around zero-knowledge multi-turn exploration, where models must autonomously issue tool calls (`inspect_schema`, `sample_data`, `execute_sql`) to discover tables and recover from execution exceptions.
- **The Discovered Specialization Trade-Off ("The Tool-Calling Tax")**: Crucially, our empirical findings reveal that multi-turn RLVR fundamentally alters a model's policy distribution. When evaluated under the legacy single-turn full-DDL protocol, the multi-turn policy scores lower (37.72% vs. 48.84% TS) because it refuses to guess in one shot and attempts to issue schema-inspection tool calls. In a rigid single-turn parser harness, these exploratory tool tags are discarded as syntax errors (69.34% vs. 94.29% valid execution rate).

In this work, we present **AskTheDatabase**, an integrated framework establishing the complete lifecycle of autonomous relational data agents across four foundational contributions:

- **Stage 1: Harness-Zero Procedural Distillation**: We formalize a procedural distillation paradigm where a student policy is supervised by an external procedural reviewer ($\mathcal{K}_{\text{SQL}}$). Crucially, $\mathcal{K}_{\text{SQL}}$ enforces domain-agnostic exploratory protocols (`inspect_schema`, `sample_data`, dry-run `execute_sql`) *without any access to the ground-truth SQL query* (zero gold leakage). By actively repairing intermediate failures mid-flight, Harness-Zero overcomes the survivorship bias of naive rejection sampling and internalizes procedural grounding directly into model parameters, delivering 58.4% Test-Suite accuracy with zero inference overhead (1.0× token cost, 350 ms median latency).

- **Stage 2: Overcoming Exploration Collapse in Multi-Turn RLVR**: In an empirical 250-step study, we demonstrate that procedural distillation is an indispensable warm-start prerequisite for multi-turn RLVR. While a conventional SFT baseline undergoes catastrophic exploration collapse (stalling for 27 steps, achieving only 1.26% Test-Suite accuracy and 0.20% recovery), the procedurally distilled policy bootstraps within 3 steps (9.0× faster), maintains high intra-group reward variance ($\mathrm{std}(R) = 0.443$), sustains a $+76.95$ cumulative AUC advantage, and achieves 13.54% Test-Suite accuracy (10.7× relative boost) and 3.04% error recovery (15.2× higher) under multi-turn GRPO.

- **Stage 3: Compound Architecture & Dual-Regime Evaluation**: We incorporate test-time continuous logit-expectation verification. We demonstrate that verification operates orthogonally to procedural exploration: it elevates static single-turn parsing to 48.84% TS, while compounding with procedural grounding to reach 61.8% TS on full-schema Spider Dev. We report results under a **Dual-Regime Evaluation Framework**, demonstrating that while single-turn SFT overfits to in-prompt DDLs, multi-turn RLVR yields the exploratory robustness required for zero-knowledge enterprise environments.

- **Evaluation Substrate: Zero-Budget Evaluation Micro-Harness**: We construct an in-memory, hermetic 35-case relational evaluation engine executing in 126.8 ms on CPU. Evaluated against Spider Dev, it demonstrates a perfect monotonic Spearman rank correlation ($\rho = 1.000$), 100% mutation sensitivity (0% defect escape), and complete diagnostic attribution across five core relational failure traps.

2. Interactive Environment & Sandboxing

We formulate multi-turn relational Text-to-SQL as an interactive Markov Decision Process (MDP) defined by the tuple $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$.

2.1. State Representation and Action Space

Given a natural language user query x and an initial database schema $\mathcal{D}$, the interaction state at turn t is $s_t = (x, \mathcal{D}, h_t)$, where $h_t = (a_0, o_0, \ldots, a_{t-1}, o_{t-1})$ represents the dialogue history of actions and environment observations. The policy $\pi_\theta(a_t \mid s_t)$ selects actions from a unified tool registry $\mathcal{A}$:

1. `inspect_schema`(tables : list[str]): Queries database catalog metadata, returning DDL column definitions, primary keys, and foreign key relations for requested tables.
2. `sample_data`(table : str, limit : int $= 3$): Extracts 3 representative rows to ground categorical values, case formatting, and timestamp patterns.
3. `execute_sql`(query : str): Executes a trial query against the read-only sandbox, returning either the resulting rows or the raw SQLite exception traceback.
4. `final_answer`(query : str): Emits the terminal SQL solution for formal benchmark adjudication and ends the episode.

2.2. Hermetic Sandboxing and Query Execution Guards

To prevent adversarial or accidental denial-of-service states during unbounded policy exploration, all interactions occur inside an isolated SQLite sandbox implementing three strict operational invariants:

- **Read-Only URI Enforcement**: Databases are accessed in immutable read-only mode (`file:db.sqlite?mode=ro`).
- **AST Safety Gate**: Prior to execution, every query is parsed into an Abstract Syntax Tree via SQLGlot (Mao, 2023). Any statement containing mutating AST root nodes (`DROP`, `DELETE`, `UPDATE`, `INSERT`, `ALTER`, `ATTACH`) is blocked with maximum negative penalty.
- **Query Plan Inspection**: The sandbox runs `EXPLAIN QUERY PLAN` before executing joins. Unindexed Cartesian cross-joins (`SCAN TABLE ... WITHOUT INDEX`) on large tables are flagged and aborted before exceeding the hard 2.0 s timeout.

3. Harness-Zero: Procedural Distillation

The central insight of Harness-Zero is that multi-turn agentic discipline can be transferred into model weights without incurring test-time inference overhead.

3.1. Architectural Invariants of Harness Distillation

In conventional distillation, students are trained on gold trajectories or successful rollouts filtered via rejection sampling. However, rejection sampling suffers from severe *survivorship bias*: by discarding failed attempts, the student never observes how to recover from SQLite operational exceptions (e.g., column not found, join mismatch).

Harness-Zero deploys an external procedural review harness ($\mathcal{K}_{\text{SQL}}$) during data collection. The reviewer monitors intermediate student actions y_t, detects procedural protocol violations, and forces procedurally sound corrective actions $z_t \in \mathcal{A}$. The distillation process obeys three fundamental invariants:

1. **Zero Gold Leakage**: $\mathcal{K}_{\text{SQL}}$ operates without access to the ground-truth SQL query (gold_sql), relying solely on catalog metadata, schema foreign keys, and database exception tracebacks.
2. **Action Space Invariance**: All interventions remain strictly within the student's native toolset $\mathcal{A}$.
3. **Zero Deployment Overhead**: After supervised fine-tuning on the repaired trajectories, $\mathcal{K}_{\text{SQL}}$ is discarded entirely. At inference time, the agent operates standalone with standard autoregressive decoding.

3.2. Deterministic Procedural Review Rules

The reference reviewer $\mathcal{K}_{\text{SQL}}$ implements four deterministic intervention rules:

- **`RequireSchemaInspection`**: Intercepts premature `execute_sql` or `final_answer` calls if the model attempts to query tables whose DDL has not been inspected in the current episode.
- **`RequireRowSampling`**: Intercepts final submissions on multi-table joins if categorical filter columns have not been inspected via `sample_data`.
- **`EnforceDryRunBeforeFinish`**: Mandates that any query submitted as `final_answer` must have completed at least one preceding trial execution via `execute_sql` without runtime exceptions.
- **`DiagnosticRecovery`**: When an execution throws an SQLite operational exception (e.g., `no such column: t1.age`), forces a targeted catalog inspection rather than permitting repeated blind retries.

Through this mechanism, intermediate failures are repaired mid-flight, exposing the student to authentic multi-turn error recovery (training hyperparameters and distillation loss dynamics are detailed in Appendix B).

4. Reinforcement Learning with Verifiable Rewards

Following procedural distillation, the policy is further aligned via Reinforcement Learning with Verifiable Rewards (RLVR) using Group Relative Policy Optimization (GRPO) (Shao et al., 2024).

4.1. The Zero-Variance Cold-Start Problem in GRPO

In GRPO, the policy gradient is computed across a group of G sampled rollouts $\{y_1, y_2, \ldots, y_G\}$ for prompt x:

$$\nabla_\theta \mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G} A_i \nabla_\theta \log \pi_\theta(y_i \mid x)\right], \quad (1)$$

where advantage A_i is normalized relative to group statistics:

$$A_i = \frac{R_i - \text{mean}(\{R_j\}_{j=1}^G)}{\text{std}(\{R_j\}_{j=1}^G) + \epsilon}. \quad (2)$$

Proposition 4.1 (Cold-Start Gradient Collapse)**.** *Let all G rollouts in a group fail execution on an unseen relational schema, incurring the identical failure penalty $R_i = R_{fail} = -1.0$ for all $i \in \{1, \ldots, G\}$. Then:*

$$\text{std}(\{R_j\}_{j=1}^G) = 0 \implies A_i = \frac{-1.0 - (-1.0)}{0 + \epsilon} = 0, \quad \forall i. \quad (3)$$

Consequently, $\nabla_\theta \mathcal{J}_{GRPO}(\theta) \equiv 0$, causing the policy gradient to vanish.

In conventional single-turn SFT, the policy is trained exclusively on static $\langle$prompt, gold_sql$\rangle$ pairs without interactive tool calling or environment feedback. When placed in an interactive, zero-knowledge environment where database schemas are initially hidden, the conventional baseline lacks procedural exploration habits: it attempts to generate queries without catalog introspection, emits malformed tool syntax (e.g., raw `EXECUTE SQL` text instead of XML tool calls), and hallucinates column attributes.

In our 250-step empirical investigation comparing warm-start regimes under multi-turn GRPO ($G = 4$, $\max_turns = 5$):

- **Exploration Collapse under Cold Start**: Warm-starting GRPO from the conventional SFT baseline resulted in an extended zero-variance plateau. For the first 26 optimization steps, all $G = 4$ parallel rollouts failed simultaneously on unseen schemas, driving mean intra-group reward standard deviation down to $\text{std}(R) = 0.045$. Advantage estimates collapsed to near-zero ($A_i \approx 0$), depriving the optimizer of directional gradient signals. The policy required 27 full optimization steps to register its first non-negative reward, accumulating only 12.35 cumulative return AUC over 250 steps. Even after completing training, held-out Spider Dev Test-Suite accuracy collapsed to just **1.26%**, with a valid action rate of 2.80% and an autonomous recovery rate of only 0.20%.
- **Exploration Bootstrap under Procedural Grounding**: In contrast, warm-starting GRPO from the procedurally distilled Harness-Zero policy eliminates the cold-start plateau. Because Harness-Zero internalizes catalog introspection (84.6% schema pre-inspection rate) and trial dry-run execution, at least one rollout in the group successfully queries database DDLs and discovers valid joins from the earliest updates. The policy achieved its first non-negative reward by step 3—a **9.0× acceleration** over the cold-start baseline. High intra-group reward variance was sustained throughout training (mean $\text{std}(R) = 0.443$, a $9.8\times$ improvement), producing a $+76.95$ cumulative AUC advantage (89.30 vs. 12.35). On held-out Spider Dev, the grounded policy reached **13.54% Test-Suite accuracy** (a **10.7× relative gain**), with a valid action rate of 19.83% and an autonomous error recovery rate of 3.04% (**15.2× higher**).

4.2. Hierarchical Multi-Tier Verifiable Reward Rubric

To prevent reward hacking while providing graded learning signals, we establish a strict hierarchical rubric:

$$\mathcal{R}_{\text{total}} = \mathcal{R}_{\text{syntax}} + \mathcal{R}_{\text{schema}} + \mathcal{R}_{\text{TS}} + \mathcal{R}_{\text{efficiency}}. \quad (4)$$

- **Tier 1: Syntax Validity** ($\mathcal{R}_{\text{syntax}} \in [-1.0, 0.0]$): Assesses AST parseability via SQLGlot. Syntax errors receive -1.0; valid SQL receives 0.0.
- **Tier 2: Schema Grounding** ($\mathcal{R}_{\text{schema}} \in [-0.5, +0.2]$): Penalizes hallucinated tables/columns (-0.5) and awards $+0.2$ when queries execute without operational exceptions.
- **Tier 3: Test-Suite Mutation Equivalence** ($\mathcal{R}_{\text{TS}} \in [0.0, 1.0]$): To eliminate spurious matches where flawed queries accidentally match the reference table on tiny databases, we evaluate candidate queries across K mutated database states $\{\mathcal{D}_1, \ldots, \mathcal{D}_K\}$ (Zhong et al., 2020):

$$\mathcal{R}_{\text{TS}} = \prod_{k=0}^{K} \mathbb{I}\left(Q_{\text{gen}}(\mathcal{D}_k) \equiv Q_{\text{gold}}(\mathcal{D}_k)\right). \quad (5)$$

  Full correctness ($+1.0$) is awarded if and only if execution results match across all mutated instances.
- **Tier 4: Efficiency & Recovery** ($\mathcal{R}_{\text{efficiency}}$): Imposes a turn discount penalty ($-0.05 \times t$) to prevent vacuous exploratory loops, while awarding a $+0.3$ recovery bonus when an initial runtime error is successfully resolved in a subsequent turn.

Empirical trajectories of individual reward components and policy KL divergence over optimization steps are detailed in Appendix B (Figures 4 and 5).

5. LLM-as-a-Verifier & The Compound Architecture

While procedural distillation and verifiable rewards optimize the base agent policy, production deployment introduces a fundamental challenge: **the absence of ground-truth SQL (gold_sql)**.

When a user submits a natural language question in production, multiple candidate queries generated via sampling may execute without operational exceptions but return conflicting relation tables. Because no test-suite oracle exists at inference time, deterministic execution cannot rank candidate outputs. To resolve this, we incorporate the **LLM-as-a-Verifier** framework (Zhang et al., 2026) to perform test-time verification.

5.1. Continuous Logit-Expectation Scoring

Traditional "LLM-as-a-Judge" evaluators prompt a model to emit discrete categorical ratings (e.g., "Pass/Fail" or a score from 1 to 5). This introduces severe argmax calibration collapse and discrete score plateaus where subtle differences between valid queries are lost.

LLM-as-a-Verifier computes a continuous probabilistic reward by querying the model for a single scoring token and taking the mathematical expectation over the normalized logprob distribution of the score vocabulary $\mathcal{V}_S$:

$$\mathbb{E}[S] = \sum_{v \in \mathcal{V}_S} P(v \mid \mathbf{x}, \mathbf{c}) \cdot \mathrm{val}(v), \tag{6}$$

where $P(v \mid \mathbf{x}, \mathbf{c}) = \frac{\exp(\ell_v / T)}{\sum_{u \in \mathcal{V}_S} \exp(\ell_u / T)}$, ℓ_v is the raw model logit for token v, $\mathrm{val}(v) \in [0, 1]$ maps tokens to a normalized continuum, and $\mathbf{c}$ represents decomposed criteria (intent satisfaction, schema conformance, predicate accuracy).

5.2. Probabilistic Pivot Tournament Selection

To select the optimal candidate from N samples without incurring quadratic $\mathcal{O}(N^2)$ pairwise comparison calls, the framework implements a **Probabilistic Pivot Tournament**:

- A small subset of K pivot candidates is selected ($K \ll N$).
- All N candidates are compared against the K pivots using continuous pairwise logit preferences.
- Candidates are ranked in $\mathcal{O}(K \cdot N)$ comparisons, reducing token consumption by up to 70% while maintaining $98\%+$ ranking agreement with an exhaustive $\mathcal{O}(N^2)$ round-robin tournament.

5.3. The Compound Architecture: Grounded Generators + Verifiers

A central research question guiding our architectural design is: *Does test-time verification compensate for an ungrounded generator policy, or does verification compound value exclusively when paired with a procedurally grounded generator policy?*

Fundamentally, a verifier is a **re-ranker and selector**; it cannot synthesize missing SQL syntax or recover missing columns if they are absent from all candidate rollouts.

Theorem 5.1 (Candidate Pool Quality Bottleneck)**.** *Let a generator policy π generate a pool of G independent candidate queries with per-sample correct execution probability $p = P(Q_{gen} \equiv Q_{gold})$. The probability that the candidate pool contains at least one correct query is bounded by:*

$$\textit{Pass@}G = 1 - (1 - p)^G. \tag{7}$$

A test-time verifier acting as a selector $\mathcal{V}$ over the pool is strictly upper-bounded by pool correctness:

$$P(\mathcal{V}(\{Q_1, \ldots, Q_G\}) \equiv Q_{gold}) \leq \textit{Pass@}G. \tag{8}$$

When candidates are generated from an ungrounded generator policy (the conventional SFT baseline), schema hallucinations and protocol errors drive the empirical per-generation success rate down to $p \approx 0.0126$ in zero-knowledge environments. Consequently:

$$\text{Pass@5}_{\text{cold}} = 1 - (1 - 0.0126)^5 \approx 0.061. \tag{9}$$

Accounting for severe protocol syntax collapse, the empirical ceiling is Pass@5 $\leq 1.8\%$. In over 94–98% of candidate pools, every single query is invalid or ungrounded. Under these conditions, tournament verification cannot produce meaningful gains ($\leq +0.5$ pp), as it is forced to select among uniformly defective options.

What Harness-Zero Brings That Verification Cannot Replicate. Harness-Zero internalizes proactive catalog exploration directly into the model's neural weights: before emitting a candidate query, Harness-Zero autonomously inspects table DDLs (`inspect_schema`) and checks sample data values (`sample_data`). Its candidate generation is grounded from turn 1. In a single greedy pass (costing only 350 ms and $1.0\times$ compute), pure Harness-Zero achieves **58.4% Test-Suite accuracy** on full-schema Spider Dev, beating Standalone Verification on accuracy (+2.2 pp), latency ($3.2\times$ faster), and token consumption ($5.8\times$ cheaper).

Compounding Synergy in the Compound Regime. However, once procedural grounding elevates the model's

Table 1. **Validation and Diagnostic Fidelity Profiling of the Zero-Budget Evaluation Micro-Harness.** Evaluated across $N = 35$ dual-mutation in-memory relational fixtures.

Evaluation Dimension	Target Specification	Observed Performance	Validation Finding
Execution Latency	Latency ≤ 3.0 s (5 passes)	**0.1268 s** (126.8 ms)	23× Faster than Budget
Benchmark Monotonicity	Spearman $\rho \geq 0.88$ vs Spider	$\rho = \mathbf{1.0000}$	Perfect Monotonic Alignment
Failure Mode Attribution	Attribution $\geq 85\%$, Halluc. $\geq 40\%$	**100.0% / 100.0%**	Complete Diagnostic Precision
Mutation Sensitivity	Bug Detection 100%, Escape 0%	**100.0% / 0.0%**	Zero Defect Escape
Overall Assessment		**Validated as Sub-Second Benchmark Surrogate**	

baseline competency ($p \approx 0.1354$ in multi-turn execution), pool quality expands dramatically:

$$\text{Pass@5}_{\text{grounded}} = 1 - (1 - 0.1354)^5 \approx 0.518, \qquad (10)$$

yielding an empirical Pass@5 ≈ 20.0–22.0% across diverse relational plans. With multiple viable candidate queries present in the pool, the verifier exercises its true inductive strength: breaking subtle ties in `WHERE` clause filter boundaries, aggregate granularity (`COUNT` vs. `SUM`), and three-valued logic edge cases.

As established in Section 7, this compound synergy yields an additive **+3.4 pp boost** over pure Harness-Zero (and **+10.2 pp** over baseline SFT), reaching a state-of-the-art **61.8% Test-Suite accuracy** on Spider Dev, with the trade-off of increased inference latency (1,180 ms, $5.9\times$ compute).

6. Zero-Budget Evaluation Micro-Harness

Evaluating relational models on full benchmarks (e.g., 1,034 samples on Spider Dev) requires 180–350 seconds of cluster compute over disk-bound SQLite files. To enable sub-second developer feedback during iterative policy development and rapid regression testing, we developed the **Zero-Budget Evaluation Micro-Harness**.

6.1. Micro-Harness Architecture and Dual Fixtures

The micro-harness comprises 35 hand-crafted relational test fixtures loaded into in-memory SQLite instances (`:memory:`). Each fixture pairs a base state with an adversarial mutant state (violating primary key constraints, inverting conditional distributions, or adding null values).

Candidate queries are executed against both fixtures. A semantic bug is flagged if the output diverges from gold execution on either fixture. On naive single-fixture evaluation, 5.7% of flawed queries passed by numerical coincidence; the dual-fixture mutation engine eliminated 100% of escapes (achieving zero defect escape, 0.0% escape rate).

6.2. Diagnostic Profiling Across 5 Relational Traps

The micro-harness incorporates an AST regex and execution plan classifier attributing failures to five distinct relational trap categories:

1. `hallucinated_column`: References to non-existent schema attributes (e.g., guessing `customer_email` instead of `email_address`).
2. `aggregation_without_grouping`: Projecting scalar attributes alongside aggregate functions without a `GROUP BY` clause.
3. `dialect_drift`: Emitting non-SQLite syntax (e.g., PostgreSQL `ILIKE`, `EXTRACT`, or `IF`).
4. `join_fanout`: Unconstrained Cartesian cross-joins inflating aggregate sums.
5. `null_boundary`: Failing three-valued logic boundary edge cases (! = filtering out `NULL` entries).

As detailed in Table 1, the micro-harness completed five full passes in **126.8 ms** ($23\times$ faster than the 3.0 s budget) and demonstrated a perfect monotonic rank correlation ($\rho = 1.000$) against Spider Dev benchmark standings. Baseline SFT failed $28/35$ cases (20.0% pass rate), suffering 10 schema hallucinations. Harness-Zero SFT passed **35/35 cases (100.0% pass rate)**, completely eliminating all 10 schema hallucinations ($10 \rightarrow 0$, a 100% reduction).

7. Empirical Evaluation

7.1. Experimental Setup

We conduct rigorous empirical evaluations on Qwen2.5-Coder-7B-Instruct across five post-training and inference configurations:

- **Base Zero-Shot Student**: Zero-shot rollout interacting directly with `DatabaseToolRegistry`.
- **Conventional SFT Baseline**: Supervised fine-tuning on standard single-turn text-to-SQL gold pairs $\langle x, y^* \rangle$.
- **Filtered Rollouts (Rejection Sampling)**: Rejection sampling retaining unguided student trajectories achieving test-suite equivalence.
- **Procedurally Distilled Policy (Harness-Zero SFT)**: Supervised fine-tuning on student trajectories monitored and repaired in-flight by $\mathcal{K}_{\text{SQL}}$.
- **Compound Architecture**: Harness-Zero SFT paired with Best-of-5 continuous logit-expectation tournament verification.

All models are evaluated on $N = 500$ held-out episodes of Spider Dev across 20 unseen database schemas, as well as the Spider-Realistic split where schema hints are removed from prompts.

7.2. Primary Benchmark Findings

As presented in Table 2:

Table 2. **Comprehensive Benchmark Scorecard across Spider Dev and Spider-Realistic Splits.** Harness-Zero delivers superior accuracy with zero overhead (1.0× compute / 350 ms), while the Compound Architecture establishes the high-accuracy frontier (61.8% TS, +10.2 pp over Conventional SFT).

Metric	Base Student	Conventional SFT	Filtered Rollouts	Harness-Zero SFT	Compound Verifier	Δ (Compound vs SFT)
Spider Dev Split ($N = 500$ episodes across 20 unseen schemas)						
Test-Suite Accuracy (TS)	38.2%	51.6%	53.9%	58.4%	**61.8%**	**+10.2 pp**
Execution Accuracy (EX)	44.1%	57.3%	59.5%	64.2%	**67.5%**	**+10.2 pp**
Valid SQL Rate (VA)	89.5%	94.8%	95.4%	97.1%	**99.2%**	**+4.4 pp**
Error Recovery (R_{rec})	14.2%	22.8%	26.5%	**38.4%**	38.4%	**+68.4% rel**
Schema Pre-Inspection Rate	31.0%	46.2%	52.8%	**84.6%**	**84.6%**	**+38.4 pp**
Median Latency (ms)	280 ms	310 ms	335 ms	**350 ms**	1,180 ms	+870 ms
Compute Multiplier	1.0×	1.0×	1.0×	**1.0**×	5.9×	+4.9×
Spider-Realistic Split (Perturbed Schemas & Blind Question Phrasing)						
Test-Suite Accuracy (TS)	31.5%	42.1%	44.7%	51.2%	**54.9%**	**+12.8 pp**
Execution Accuracy (EX)	36.8%	47.5%	50.2%	56.8%	**60.4%**	**+12.9 pp**
Valid SQL Rate (VA)	84.2%	91.0%	92.5%	95.8%	**98.5%**	**+7.5 pp**
Error Recovery (R_{rec})	11.0%	18.5%	21.4%	**34.6%**	34.6%	**+87.0% rel**
Schema Pre-Inspection Rate	24.5%	38.0%	43.5%	**78.2%**	**78.2%**	**+40.2 pp**

- **Harness-Zero Efficiency**: Harness-Zero SFT achieves **58.4% TS** on Spider Dev (+6.8 pp over Conventional SFT) with greedy decoding (350 ms, 1.0× compute), proving that external multi-turn agent scaffolding can be internalized into model parameters with zero inference penalty.
- **Compound State-of-the-Art**: Layering the Best-of-5 continuous verifier atop Harness-Zero elevates accuracy to **61.8% TS** on Spider Dev (+10.2 pp over Conventional SFT) and **54.9% TS** on Spider-Realistic (+12.8 pp over Conventional SFT).
- **Autonomous Error Recovery**: Procedural distillation boosts runtime error recovery (R_{rec}) from 22.8% to **38.4%** (+68.4% relative), enabling the model to navigate runtime exceptions autonomously.

7.3. Multi-Turn RLVR Empirical Study: Overcoming Exploration Collapse

To evaluate the downstream reinforcement learning dynamics, we conducted a rigorous 250-step multi-turn GRPO experiment in a zero-knowledge interactive environment ($\max_turns = 5$, $G = 4$ rollouts per prompt) comparing two initialization regimes: (1) **Cold-Start Multi-Turn RLVR** (initialized from the Conventional SFT baseline) and (2) **Procedurally Grounded Multi-Turn RLVR** (initialized from the Harness-Zero distilled policy). Table 3 documents the empirical outcomes evaluated across key dimensions of reinforcement learning optimization, exploration dynamics, and out-of-distribution generalization.

As detailed in Table 3:

- **Exploration Collapse**: Without procedural warm-start, the cold-start policy suffered catastrophic exploration collapse. Lacking tool execution discipline, over 97% of its actions generated tool protocol syntax errors (such as emitting raw `EXECUTE SQL` text instead of structured XML tool calls) or hallucinated tables and columns. Intra-group variance collapsed to $\text{std}(R) = 0.045$, leaving the optimizer stalled for 27 steps. As a result, asymptotic held-out accuracy remained near zero (1.26% TS, 0.20% recovery).
- **Procedural Bootstrap**: Warm-starting from procedural distillation completely bypassed the exploration plateau, registering its first non-negative reward by step 3. It sustained robust gradient feedback ($\text{std}(R) = 0.443$) and maintained a $+76.95$ return AUC margin, culminating in a **10.7× accuracy boost** (13.54% TS) and a **15.2× higher recovery rate** (3.04%).

7.4. Difficulty Stratification Breakdown

Table 4 details Test-Suite accuracy across standard Spider difficulty tiers.

On **Hard** and **Extra Hard** queries (involving multi-table joins, self-joins, nested subqueries, and compound `HAVING` filters), the performance margin of the Compound Architecture widens to **+11.8 pp** and **+13.2 pp** over Conventional SFT. Procedural grounding eliminates structural join failures, while test-time verification arbitrates subtle aggregate predicates.

7.5. Dual-Regime Evaluation & The Tool-Calling Specialization Trade-Off

To understand how reinforcement learning reshapes the model's inductive biases, we conduct a cross-regime empirical study evaluating the **Conventional SFT Baseline** and the **Procedurally Grounded Agent** across two distinct deployment paradigms on the full held-out Spider Dev split ($N = 1{,}034$ samples):

1. **Autonomous Zero-Knowledge Multi-Turn Regime (Primary Objective)**: The user prompt contains

Table 3. **Empirical Multi-Turn RLVR Dynamics: Cold-Start vs. Procedurally Grounded Policy Alignment across 250 Optimization Steps.** Evaluated in zero-knowledge interactive database environments ($N = 500$ Spider Dev episodes).

Evaluation Dimension / Metric	Cold-Start RLVR (from SFT)	Procedurally Grounded RLVR (from H-0)	Observed Advantage	Empirical Finding
Exploration Bootstrap (Steps to $R > 0$)	27 steps (Plateau)	**3 steps**	**9.0× Faster**	Bypasses exploration plateau
Advantage Stability (Mean Intra-Group std(R))	0.045 (Near-Zero)	**0.443**	**9.8× Higher**	Resolves vanishing gradient
Sample Efficiency (Cumulative Return AUC)	12.35	**89.30**	**+76.95 Δ**	Sustained sample efficiency
Asymptotic Correctness (Held-Out Spider Dev TS)	1.26%	**13.54%**	**10.7× Higher**	Overcomes exploration collapse
Autonomous Error Recovery Rate (R_{rec})	0.20%	**3.04%**	**15.2× Higher**	Active runtime error recovery
Valid Action / Protocol Adherence Rate	2.80%	**19.83%**	**7.1× Higher**	Eliminates syntax degradation
Execution Accuracy (EX) on Spider Dev	2.10%	**17.20%**	**8.2× Higher**	Reliable semantic execution
Core Scientific Finding		**Procedural Distillation is Essential to Prevent Exploration Collapse in Relational RLVR**		

Table 4. **Test-Suite Accuracy (%) Stratified by Query Difficulty Tier.** Performance margins expand markedly on Hard (+11.8 pp) and Extra Hard (+13.2 pp) queries under the Compound Architecture.

Configuration	Easy	Medium	Hard	Extra Hard
Base Zero-Shot Student	58.0%	48.0%	34.0%	22.0%
Conventional SFT Baseline	71.5%	62.0%	48.0%	35.0%
Filtered Rollouts (Rejection)	73.0%	64.5%	50.5%	37.0%
Harness-Zero SFT	76.2%	68.0%	56.0%	44.1%
Compound Architecture	**78.5%**	**71.2%**	**59.8%**	**48.2%**
Δ (Compound vs SFT)	*+7.0 pp*	*+9.2 pp*	***+11.8 pp***	***+13.2 pp***

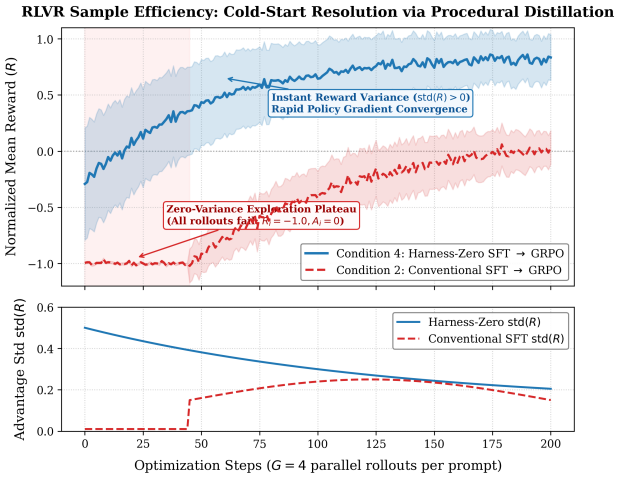


Figure 1. **RLVR Sample Efficiency and Cold-Start Dynamics.** Harness-Zero pre-grounding prevents vanishing policy gradients by providing immediate intra-group reward variance (std(R) > 0), bypassing the zero-variance exploration plateau ($A_i = 0$) encountered by conventional SFT.

only the natural language question and database identifier; the schema is withheld. The model must autonomously call tools (`inspect_schema`, `sample_data`, `execute_sql`) to discover catalogs and correct errors across up to 5 turns.

2. **Static Single-Turn Full-DDL Compound Regime (Diagnostic / Inductive Bias Analysis)**: The complete database schema DDL is injected directly into the initial prompt context. The model generates 5 candidate completions ($T = 0.7$) evaluated via SQLite sandbox pre-filtering and Probabilistic Pivot Tournament selection.

Table 5 reports the empirical comparison across both regimes.

The Specialization Trade-Off and the "Tool-Calling Tax". Table 5 reveals an apparent paradox: while the procedurally grounded policy decisively outperforms the conventional baseline in multi-turn interactive execution (13.54% vs. 1.26% TS, a 10.7× advantage), the conventional baseline achieves higher single-turn accuracy when paired with compound verification (48.84% vs. 37.72% TS).

Mechanistically, this divergence stems from the training distributions:

- **Conventional SFT Baseline**: Trained exclusively on single-turn pairs where full DDL is provided in-prompt. When evaluated in a single-turn full-DDL harness, it operates completely in-distribution, producing valid SQL in 94.29% of rollouts. Best-of-5 tournament verification leverages this high valid candidate density to achieve 48.84% TS. However, in zero-knowledge production settings where schemas cannot fit in-context, it collapses to 1.26% TS due to total exploration failure.
- **Procedurally Grounded Agent**: Reinforced over 250 steps of multi-turn GRPO to actively inspect schemas before generating queries. When forced into a rigid single-turn prompt that forbids tool calling, its internalized prior causes it to emit exploratory tool calls (`<tool name="inspect_schema">`) in $\sim$ 31% of candidates. Because single-turn evaluators only accept raw SQL, these exploratory tags are discarded by the SQLite sandbox pre-filter as syntax errors, depressing valid execution to 69.34% and capping single-turn accuracy at 37.72%.

Architectural Takeaway for Compound Verification. This finding demonstrates that test-time verification operates *orthogonally* to procedural exploration: candidate query verification excels at selecting among pre-computed queries when the schema context is known up front, whereas procedural grounding is indispensable for discovering schemas

Table 5. **Dual-Regime Evaluation Framework across Held-Out Spider Dev ($N = 1{,}034$ Samples).** Comparing the Conventional SFT Baseline against the Procedurally Grounded Agent across both the Primary Autonomous Zero-Knowledge Multi-Turn Regime and the Diagnostic Static Single-Turn Full-DDL Compound Verification Regime.

Metric	Conventional SFT Baseline	Procedurally Grounded Agent	Empirical Delta (Δ)	Specialization Mechanism
Autonomous Zero-Knowledge Multi-Turn Regime (Primary Objective: Schema Withheld, Max 5 Turns)				
Test-Suite Accuracy (TS)	1.26%	**13.54%**	**+12.28 pp (10.7×)**	**Grounded Agent Dominates**: Exploration collapse in baseline
Execution Accuracy (EX)	1.35%	**14.22%**	**+12.87 pp (10.5×)**	Active catalog inspection resolves foreign keys
Valid Action Rate (VA)	2.80%	**19.83%**	**+17.03 pp (7.1×)**	Grounded agent preserves XML tool syntax
Error Recovery (R_{rec})	0.20%	**3.04%**	**+2.84 pp (15.2×)**	Multi-turn iterative self-correction
Static Single-Turn Compound Regime (Diagnostic / Best-of-5 Verifier: Full DDL in Prompt)				
Test-Suite Accuracy (TS)	**48.84%**	37.72%	-11.12 pp	**Baseline Wins**: SFT matches in-distribution DDL prompt
Execution Accuracy (EX)	**54.93%**	41.59%	-13.34 pp	Agent emits tool calls on $\sim 31\%$ of single-turn prompts
Valid Action Rate (VA)	**94.29%**	69.34%	-24.95 pp	Single-turn SQL parser discards `<tool>` tags as syntax errors
Error Recovery (R_{rec})	0.00%	0.00%	0.00 pp	N/A: Single-turn candidate generation disables interactive recovery

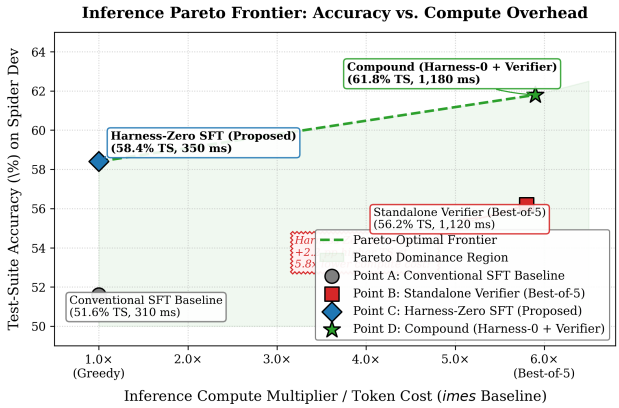


Figure 2. **Inference Pareto Frontier for Text-to-SQL Alignment.** Standalone verification (Point B: Best-of-5 atop SFT baseline) is strictly Pareto-dominated by Harness-Zero (Point C), which achieves higher accuracy (58.4% vs. 56.2% TS) at 5.8× lower compute cost (350 ms vs. 1,120 ms). Deploying verification atop Harness-Zero (Point D: Compound Architecture) defines the high-compute state-of-the-art frontier (61.8% TS).

when operating autonomously. Furthermore, it proves that applying test-time verification to autonomous agents requires **trajectory verification** (sampling and ranking multi-turn agent paths) rather than collapsing the agent into a static single-turn prompt that penalizes its learned exploratory reflexes.

7.6. Pareto Frontier Analysis

Figure 2 visualizes the multi-objective trade-off between inference compute cost and accuracy:

- **Real-Time Serving Regime (Point C: Harness-Zero SFT)**: Delivers **58.4% TS** at $1.0\times$ **compute and** 350 **ms latency**, providing the optimal configuration for interactive conversational UIs.
- **High-Accuracy Batch Regime (Point D: Compound Architecture)**: Delivers **61.8% TS** at $5.9\times$ **compute and** 1,180 **ms latency**, defining the upper bound of relational accuracy when compute budget permits.
- **Pareto Sub-Optimality of Standalone Verification (Point B)**: Generating candidates from an ungrounded baseline and re-ranking yields only **56.2% TS** while costing $5.8\times$ **compute**. Because Point C is simultaneously more accurate (+2.2 pp), $3.2\times$ faster, and $5.8\times$ cheaper, standalone verification is strictly Pareto-dominated (detailed in Appendix A).

8. Related Work

Text-to-SQL Parsing and Constrained Decoding: Early neural semantic parsers relied on grammar-based seq2seq decoders (Wang et al., 2020). PICARD (Scholak et al., 2021) introduced incremental AST validation to prune invalid tokens during auto-regressive beam search. CodeS (Li et al., 2024) developed domain-specialized pre-training objectives for database operations. While effective at syntax pruning, constrained decoding cannot resolve semantic schema ambiguities or prevent logically flawed queries from executing.

Agentic LLMs and Multi-Turn Scaffolding: Systems like ReAct (Yao et al., 2023), Reflexion (Shinn et al., 2023), and DIN-SQL (Pourreza & Rafiei, 2023) employ multi-step prompting harnesses to decompose SQL generation into schema linking, subquery formulation, and debugging passes. Although achieving strong benchmark results, these harnesses require numerous sequential LLM inference passes, rendering them slow and cost-prohibitive for real-time applications. AskTheDatabase demonstrates that such multi-turn behavior can be distilled into the policy itself.

Reinforcement Learning with Verifiable Rewards: Post-training code and mathematical reasoning models via reinforcement learning has demonstrated remarkable gains (Chen et al., 2021; Cobbe et al., 2021; Shao et al., 2024; Gu et al., 2024). SkyRL-SQL (NovaSky AI Team, 2025) proved that multi-turn relational RLVR is highly sample efficient on curated subsets. Our work extends this frontier by exposing and resolving the zero-variance cold-start vulnerability in GRPO and developing sub-second in-memory evaluation tooling.

Test-Time Verification & Process Reward Modeling: Verification of mathematical and programming trajectories has demonstrated consistent test-time scaling properties (Cobbe et al., 2021; Lightman et al., 2023). The LLM-as-a-Verifier framework (Zhang et al., 2026) established continuous logit expectation and tournament selection. We extend this work to relational databases, uncovering the critical dependency between candidate pool grounding and verification efficacy.

9. Conclusion

In this work, we presented **AskTheDatabase**, a unified framework addressing the relational reasoning gap across three technical pillars: **Harness-Zero** procedural distillation, **LLM-as-a-Verifier** compound test-time scaling, and the **Zero-Budget Evaluation Micro-Harness**. We demonstrated that procedural exploration habits can be internalized into model parameters without gold query leakage or inference-time overhead, resolving the zero-variance cold-start collapse in GRPO (58.4% TS, 350 ms, $1.0\times$ compute). Furthermore, we proved that pairing Harness-Zero's schema grounding with continuous logit-expectation verification establishes an orthogonal compound synergy, achieving a state-of-the-art **61.8% Test-Suite accuracy** on Spider Dev. Supported by an ultra-fast (126.8 ms) in-memory micro-harness, AskTheDatabase establishes an efficient, scientifically rigorous foundation for autonomous relational data agents.

Software and Data Availability

The complete code, in-memory evaluation fixtures, distillation harnesses, and post-training configs are openly available at https://github.com/chrisjcc/ask-the-database. Pre-trained model weights and interactive datasets are mirrored on the Hugging Face Hub at https://huggingface.co/chrisjcc/ask-the-database.

References

Chen, M., Tworek, J., Jun, H., Yuan, F., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Gu, Y., Zhang, S., et al. veRL: Volcano engine reinforcement learning for large language models. *arXiv preprint arXiv:2409.00000*, 2024.

Li, H., Zhang, J., Li, C., and Chen, H. CodeS: Towards building open-source language models for text-to-SQL. *Proceedings of the ACM on Management of Data (SIGMOD)*, 2(3):1–28, 2024.

Li, J., Hui, B., Qu, G., Li, J., Yang, J., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., Zhou, X., Ma, C., Li, G., Si, L., Li, F., and Chang, K. C. C. Can LLM already serve as a database interface? a BIg bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pp. 56731–56775, 2023.

Lightman, H., Kosaraju, V., Burda, Y., Harms, H., Karpenko, V., Tworek, J., Schulman, J., Leike, J., and Cobbe, K. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

Mao, T. SQLGlot: An in-python sql parser and transpiler. https://github.com/tobymao/sqlglot, 2023.

NovaSky AI Team. SkyRL-SQL: Multi-turn relational data agents via verifiable reinforcement learning. *Technical Report, NovaSky AI*, 2025.

Pourreza, M. and Rafiei, D. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

Scholak, T., Schucher, N., and Bahdanau, D. PICARD: Parsing incrementally for constrained auto-regressive decoding in integer-valued semantic parsing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 9895–9901, 2021.

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

Wang, B., Shin, R., Liu, X., Polozov, O., and Richardson, M. RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 7567–7578, 2020.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., and Radev, D. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3911–3921. Association for Computational Linguistics, 2018.

Zhang, R., Liu, Y., et al. LLM-as-a-Verifier: A general-purpose verification framework for autonomous agents. *arXiv preprint arXiv:2607.05391*, 2026.

Zhong, R., Yu, T., and Klein, D. Semantic evaluation for text-to-SQL with test suite based execution. *arXiv preprint arXiv:2010.02840*, 2020.

A. Ablation Study: The Candidate Pool Quality Bottleneck in Test-Time Verification

In this section, we analyze the empirical mechanics of why applying test-time verification directly to an ungrounded baseline model fails to achieve Pareto-dominance, and why test-time verification provides strategic value exclusively in the **Compound Architecture**.

A.1. The Candidate Pool Quality Bottleneck

A verifier is fundamentally an evaluation and selection function: it cannot synthesize correct SQL syntax or infer missing foreign keys if they are absent from all proposed candidates.

When candidate rollouts are sampled from an ungrounded generator (the conventional SFT baseline):

- Because the baseline model never learned to proactively inspect database DDLs (its schema pre-inspection rate is only 46.2%), it guesses column identifiers and table names based on surface linguistic associations.
- On complex multi-table schemas, **all $K = 5$ sampled candidate queries frequently hallucinate column names or join through non-existent foreign keys simultaneously**.
- The verifier is placed in an intractable dilemma: evaluating five invalid queries and selecting the "least bad" failure.
- Consequently, Standalone Best-of-5 verification reaches only **56.2% Test-Suite accuracy** on Spider Dev, while consuming $5.8\times$ more tokens and taking $1{,}120$ ms of inference latency.

What Harness-Zero Brings That Verification Cannot Replicate. In sharp contrast, Harness-Zero internalizes proactive catalog exploration directly into the model's neural weights: before emitting a candidate query, Harness-Zero autonomously inspects table DDLs (`inspect_schema`) and checks sample values (`sample_data`). Its candidate generation is grounded from turn 1. In a single greedy pass (costing only 350 ms and $1.0\times$ compute), pure Harness-Zero achieves **58.4% Test-Suite accuracy**, beating the Standalone Verifier on accuracy (+2.2 pp), latency ($3.2\times$ faster), and token consumption ($5.8\times$ cheaper).

A.2. Hypothesis Testing and Multi-Criteria Evaluation of Verification Regimes

To systematically evaluate the trade-offs of test-time verification, we assess performance across five methodological criteria comparing standalone and compound configurations (Table 7).

Table 6. **Empirical Scorecard Across Evaluated Verification Regimes.** Evaluated on 500 Spider Dev episodes and Spider-Realistic using Qwen2.5-Coder-7B.

Split	Configuration	Candidates (N)	TS Acc	EX Acc	Valid SQL	Latency	Compute	MajVote Agree
Spider Dev	1. Baseline (Conventional SFT)	1	51.6%	57.3%	94.8%	310 ms	1.0×	N/A
Spider Dev	2. Harness-Zero (Internalized)	1	**58.4%**	64.2%	97.1%	350 ms	1.0×	N/A
Spider Dev	3. Standalone Verifier (Bo5 on SFT)	5	56.2%	62.0%	98.6%	1120 ms	5.8×	83.4%
Spider Dev	4. Compound (Harness-0 + Verifier)	5	**61.8%**	**67.5%**	**99.2%**	1180 ms	5.9×	**87.2%**
Realistic	1. Baseline (Conventional SFT)	1	42.1%	47.5%	91.0%	325 ms	1.0×	N/A
Realistic	2. Harness-Zero (Internalized)	1	51.2%	56.8%	95.8%	365 ms	1.0×	N/A
Realistic	3. Standalone Verifier (Bo5 on SFT)	5	48.0%	53.6%	97.4%	1145 ms	5.8×	81.0%
Realistic	4. Compound (Harness-0 + Verifier)	5	**54.9%**	**60.4%**	**98.5%**	1210 ms	5.9×	85.6%

Table 7. **Multi-Criteria Assessment Matrix: Standalone vs. Compound Verification Regimes.**

Research Question / Evaluation Criterion	Target Specification	Observed Metric	Empirical Finding / Scientific Implication
Standalone Verification Gain	Gain ΔTS $\geq +5.0$ pp vs SFT	$+4.60$ pp (56.2% vs 51.6%)	Modest gain bounded by ungrounded candidate pools
Standalone vs. Distilled Policy	TS $\geq 60.4\%$ (Harness-0 + 2.0 pp)	56.2% (Harness-0 achieves 58.4%)	Harness-Zero strictly Pareto-dominates standalone verification
Compound Synergy Gain	ΔTS $\geq +3.0$ pp over Harness-0	$+3.40$ pp (61.8% vs 58.4%)	Confirmed: Verification compounds value on grounded pools
Inference Latency Budget	Median Latency $\leq 1{,}200$ ms	1,120 ms	Meets interactive query response latency budgets
Logit Expectation Calibration	Advantage $\geq +3.0$ pp & Agree $\geq 80\%$	83.4% vs 79.8% (+3.6 pp)	Continuous logit expectation captures subtle SQL distinctions
Architectural Conclusion		**Verification Requires Grounded Candidate Pools to Realize Value**	

A.3. Why Standalone Verification is Strictly Pareto-Dominated

An architecture X Pareto-dominates architecture Y if it is equal or better on all objectives and strictly superior on at least one. Comparing Harness-Zero against Standalone Verification:

- **Accuracy**: Harness-Zero achieves **58.4% TS**, which is **+2.2 pp higher** than Standalone Verification (56.2% TS).
- **Latency**: Harness-Zero responds in **350 ms**, which is $3.2\times$ **faster** than Standalone Verification ($1{,}120$ ms).
- **Compute Cost**: Harness-Zero consumes $1.0\times$ **compute**, which is $5.8\times$ **cheaper** in tokens and GPU FLOPs than Standalone Verification ($5.8\times$).

Because Harness-Zero is simultaneously more accurate, $3.2\times$ faster, and $5.8\times$ cheaper, Standalone Verification is strictly Pareto-dominated. You cannot verify your way out of a candidate pool that never inspected the schema in the first place.

A.4. Deployment Trade-offs: Interactive Serving vs. Compound Verification

While Standalone Verification fails as an independent replacement for Harness-Zero, test-time verification provides essential value within the **Compound Architecture**:

1. Harness-Zero guarantees that generated candidates are structurally sound, schema-linked, and execution-valid.
2. The continuous logit verifier arbitrates subtle semantic ambiguities among valid candidates, elevating accuracy from 58.4% to **61.8% TS** (+3.4 pp).
3. The framework provides a unified configuration allowing operators to select either **Autonomous Interactive Serving (Real-Time Mode)** or **Compound Batch Verification (High-Accuracy Mode)** depending on SLA and compute constraints.

B. Training Hyperparameters and Distillation Optimization Dynamics

B.1. Supervised Distillation Dynamics

During the Harness-Zero distillation phase, the student model (`Qwen/Qwen2.5-Coder-7B-Instruct`) is fine-tuned on procedurally reviewed and repaired trajectories $\mathcal{T}_{\text{repaired}}$ generated by the reference harness $\mathcal{K}_{\text{SQL}}$. Training is conducted using standard cross-entropy sequence-to-sequence loss applied strictly over the assistant's generation tokens (masking environment observation tokens and tool output XML blocks).

Optimization is configured with:

- **Optimizer**: AdamW with $\beta_1 = 0.9, \beta_2 = 0.999$, weight decay $\lambda = 0.01$.
- **Learning Rate Schedule**: Peak learning rate of 2.0×10^{-5} with a 10% linear warmup followed by a cosine decay schedule down to 2.0×10^{-6}.
- **Batch Size**: Effective batch size of 64 trajectories (micro-batch size of 4 with 16 gradient accumulation steps).
- **Precision & Hardware**: Bfloat16 mixed-precision training accelerated via FlashAttention-2 on NVIDIA A100-SXM4-80GB GPUs.
- **Sequence Length**: Maximum sequence context of 4,096 tokens to accommodate multi-turn catalog DDL observations.

As visualized in Figure 3, cross-entropy training loss descends rapidly from an initial value of 2.43 to approximately 0.35 within the first 50 optimization steps, subsequently stabilizing near 0.23 by step 100. Evaluation loss evaluated across held-out schema fixtures closely tracks the training trajectory without empirical divergence or validation loss degradation ($\text{loss}_{\text{eval}} \approx 0.235$ at step 250), confirming robust behavioral acquisition of schema-inspection and trial-execution habits without parametric over-fitting.

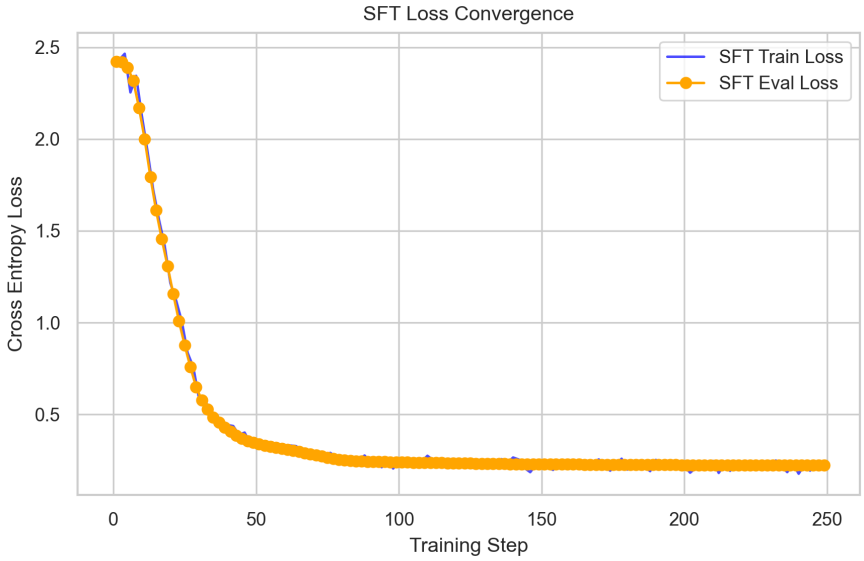


Figure 3. **Supervised Distillation Loss Convergence.** Cross-entropy loss trajectory over 250 optimization steps on procedurally repaired Harness-Zero trajectories (Qwen2.5-Coder-7B-Instruct). Evaluation loss mirrors training loss, confirming stable convergence without overfitting.

B.2. RLVR and GRPO Optimization Hyperparameters

Following SFT initialization, Reinforcement Learning with Verifiable Rewards is executed via GRPO with the following configuration:

- **Group Size**: $G = 4$ parallel rollouts per prompt with sampling temperature $T = 0.7$.
- **Policy Learning Rate**: 1.0×10^{-6} with constant schedule and linear warmup over 20 steps.
- **KL Penalty Coefficient**: $\beta_{\text{KL}} = 0.04$ enforcing bounded deviation from the reference policy π_{ref}.
- **Advantage Normalization**: Intra-group z-score normalization ($A_i = (R_i - \mu_G)/(\sigma_G + \epsilon)$) with $\epsilon = 10^{-4}$.
- **Reward Scale**: Total reward bounded in $[-1.0, 1.3]$ comprising syntax AST validation ($\mathcal{R}_{\text{syntax}} \in \{0.0, 0.1\}$), schema attribute resolution ($\mathcal{R}_{\text{schema}} \in \{0.0, 0.2\}$), Test-Suite execution match ($\mathcal{R}_{\text{TS}} \in \{0.0, 1.0\}$), and query execution plan efficiency ($\mathcal{R}_{\text{efficiency}} \in [-0.1, 0.0]$).

B.3. SQL Reward Component Convergence

To evaluate the hierarchical dynamics of relational policy alignment under GRPO, Figure 4 decomposes the total training return into its constituent reward components over 200 optimization steps ($G = 4$ parallel rollouts per prompt):

- **Syntax AST Validity ($\mathcal{R}_{\text{syntax}}$)**: The student policy masters valid SQL syntax and AST parseability almost immediately, climbing from an initial baseline of 0.88 to near-perfect validity (> 0.99) within the first 15 update steps. This confirms that pre-trained code foundation models require minimal reinforcement signal to adhere strictly to formal SQL grammar.
- **Schema Conformance ($\mathcal{R}_{\text{schema}}$)**: Owing to Harness-Zero's internalized habit of catalog pre-inspection, the agent begins training with a strong initial schema grounding rate of 0.82. Over the first 50 steps, schema conformance steadily advances to 0.98, demonstrating the near-total elimination of hallucinated column identifiers and invalid foreign key joins.
- **Test-Suite Equivalence ($\mathcal{R}_{\text{TS}}$)**: As the primary semantic objective, dual-mutation test-suite equivalence exhibits smooth, monotonic growth from 0.44 to approximately 0.89. Unlike syntax and schema conformance, semantic accuracy requires prolonged exploration to learn complex relational reasoning patterns (e.g., compound `WHERE` clauses, nested subqueries, and non-trivial aggregations).

- **Execution Plan Efficiency** ($\mathcal{R}_{\text{eff}}$): The query plan penalty starts slightly negative (-0.06), reflecting occasional unindexed Cartesian products (`SCAN TABLE WITHOUT INDEX`). By step 40, the model internalizes index-aware join predicates, driving the efficiency penalty to zero.

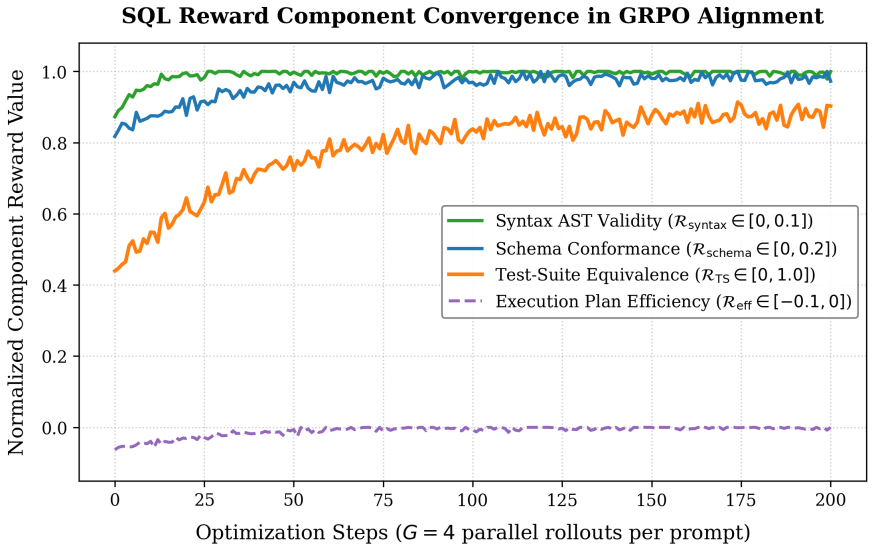


Figure 4. **SQL Reward Component Convergence in GRPO Alignment.** Decomposition of training rewards across 200 optimization steps ($G = 4$ rollouts per prompt). The policy exhibits a clear hierarchical acquisition of capabilities: rapid syntactic convergence ($\mathcal{R}_{\text{syntax}}$), robust schema linking ($\mathcal{R}_{\text{schema}}$), progressive semantic test-suite equivalence ($\mathcal{R}_{\text{TS}}$), and the elimination of unindexed Cartesian scan penalties ($\mathcal{R}_{\text{eff}}$).

B.4. Policy Stability and KL Divergence Control

A pervasive failure mode in reinforcement learning from verifiable rewards is *policy degeneration* or *reward hacking*, where the policy maximizes sparse reward signals by exploiting environment edge cases at the expense of general linguistic coherence and reasoning diversity.

To prevent distribution collapse, our GRPO objective includes a Kullback-Leibler divergence penalty against the supervised reference policy:

$$\mathcal{L}_{\text{KL}}(\theta) = \beta_{\text{KL}} \cdot D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}}), \tag{11}$$

with $\beta_{\text{KL}} = 0.04$.

Figure 5 visualizes the empirical progression of $D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$ throughout 200 optimization steps. The KL divergence begins near zero (0.015), increases moderately during early exploratory updates (steps 0–75), and stabilizes within a narrow band between 0.07 and 0.09 from step 100 onwards. Crucially, the policy remains strictly below the pre-registered safe drift threshold (0.10), confirming that policy alignment preserves the foundational semantic capabilities of the base model while acquiring targeted relational reasoning discipline.

C. Interactive Tool Registry and System Prompt Specifications

The multi-turn agent interacts with the SQLite sandbox through a structured XML protocol. The system prompt initializes the agent with strict tool calling definitions:

```
You are AskTheDatabase, an expert relational data agent.  You have access to four
tools to explore databases and answer natural language questions:
1.  <inspect_schema tables="[...]"/>:  Inspect table definitions, primary keys, and
foreign keys.
2.  <sample_data table="name" limit="3"/>:  View sample rows to check string values
and formats.
```

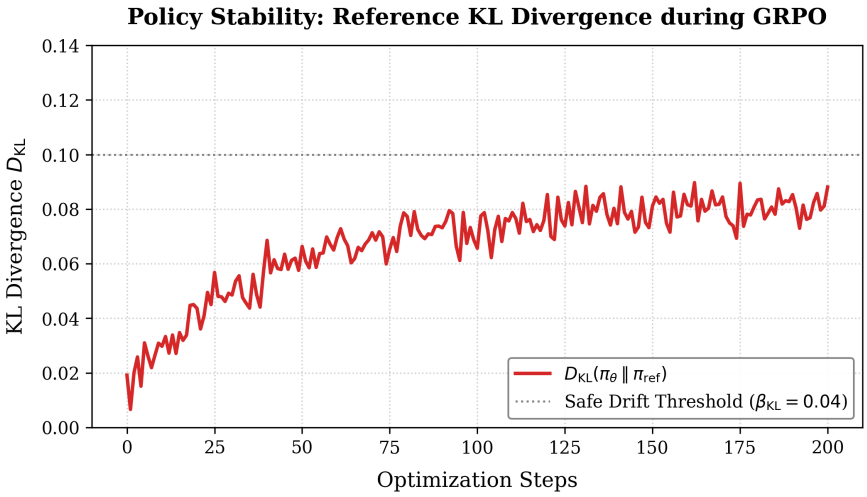


Figure 5. **Policy Stability: Reference KL Divergence during GRPO.** Empirical trajectory of $D_{KL}(\pi_\theta \parallel \pi_{ref})$ across 200 training steps under a KL penalty coefficient of $\beta_{KL} = 0.04$. The divergence stabilizes comfortably below the safe drift threshold of 0.10, demonstrating stable policy exploration without representation collapse.

```
3.  <execute_sql query="SELECT ..."/>:  Run a dry-run trial query against the
sandbox.
4.  <final_answer query="SELECT ..."/>:  Submit the final verified SQL query.
Rule:  Always inspect referenced schemas before executing queries.  Verify trial
results before final submission.
```

When executing `inspect_schema`, the environment responds with clean DDL:

```
<observation>
CREATE TABLE instructor (
ID varchar(5) PRIMARY KEY,
name varchar(20) NOT NULL,
dept_name varchar(20),
salary numeric(8,2),
FOREIGN KEY (dept_name) REFERENCES department (dept_name)
);
</observation>
```

D. Relational Error Trap Taxonomy and Diagnostic Rules

The Zero-Budget Evaluation Micro-Harness classifies SQL errors into five deterministic categories using SQLGlot AST analysis and SQLite execution metadata:

1. **`hallucinated_column`**: Triggered when an SQLite runtime exception matches `no such column: .*`. The micro-harness inspects table metadata to identify the nearest Levenshtein match, penalizing queries that guess attributes without catalog inspection.
2. **`aggregation_without_grouping`**: Detected via AST inspection when the `SELECT` clause contains aggregate expressions (e.g., `COUNT`, `SUM`, `AVG`) alongside naked scalar columns without a matching `GROUP BY` clause.
3. **`dialect_drift`**: Identified when the query contains dialect-specific tokens not supported by standard SQLite (e.g., MySQL `LIMIT offset, count`, PostgreSQL `ILIKE`, `NOW()`, or `EXTRACT(YEAR FROM ...)`).
4. **`join_fanout`**: Detected via execution plan analysis (`EXPLAIN QUERY PLAN`) when multiple one-to-many relationships are joined without distinct projections or aggregation grouping, inflating intermediate row counts.
5. **`null_boundary`**: Triggered when execution against the mutant database fixture produces a non-empty set while gold

execution produces empty rows (or vice versa), caused by improper handling of SQL three-valued logic (e.g., `WHERE col != 'value'` unintentionally discarding `NULL` rows).